

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении курсовой работы
По дисциплине: «Теоретические основы баз данных»
На тему: «Учёт компьютерных шрифтов электронной
типографии»

№ этапа	Дата	Подпись
1		
2		
3		

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
преподаватель

_____ Попов С. Г.

«_____» _____ 2026г.

Содержание

1	Аналитика	3
1.1	Описание предметной области «Учёт компьютерных шрифтов в электронной типографии»	3
1.2	Цели функционирования базы данных	6
1.3	Польза от внедрения базы данных	7
1.4	Описание сущностей	7
1.5	Описание ролей и субъектов	8
1.6	ER-диаграмма	9
1.7	Схема объектов	11
2	Проектирование	12
2.1	Лингвистическое описание схемы базы данных	12
2.2	Описание таблиц базы данных	13
2.3	Создание таблиц базы данных и генерация записей	21
3	Программирование	24
	Приложение А. SQL-скрипт создания таблиц базы данных	25
	Приложение Б. Программа на TypeScript для заполнения базы данных тестовыми данными	27

1 Аналитика

1.1 Описание предметной области «Учёт компьютерных шрифтов в электронной типографии»

Типографика — искусство оформления текста с помощью наборного шрифта — является одним из фундаментальных элементов визуальной коммуникации — передачи информации посредством зрительно воспринимаемых образов. На протяжении столетий она развивалась от ручного набора металлических литер (брусков с выпуклым изображением знака) до современных цифровых технологий. Изобретение печатного станка Иоганном Гутенбергом в середине пятнадцатого века положило начало массовому распространению печатного слова, однако процесс набора текста оставался трудоёмким ремеслом. Каждая литера отливалась из металла, а наборщик вручную составлял строки из отдельных знаков. Появление линотипа (машина, отливающая целые строки) и монотипа (машина, отливающая отдельные литеры) в конце девятнадцатого века механизировало этот процесс, но подлинная революция произошла с переходом к цифровым технологиям.

Электронная типография — организация, осуществляющая подготовку и выпуск печатной и цифровой продукции с использованием компьютерных технологий, — представляет собой современный этап развития печатного дела, в котором все процессы подготовки и воспроизведения текста осуществляются с использованием компьютерных технологий. В отличие от традиционной типографии, где шрифт существовал как физический объект, в электронной типографии шрифт является программным продуктом. Компьютерный шрифт — это файл, содержащий векторные или растровые описания графем (букв, цифр и других знаков письма), а также метрическую информацию, необходимую для корректного отображения и печати текста. Такие шрифты используются в издательских системах, графических редакторах, веб-страницах и пользовательских интерфейсах операционных систем.

В наши дни современную электронную типографию невозможно представить без обширных коллекций шрифтов. Дизайнеры, верстальщики и препресс-специалисты (специалисты по допечатной подготовке) ежедневно работают с десятками гарнитур, подбирая оптимальные решения для каждого проекта. Шрифтовая гарнитура объединяет набор начертаний, выполненных в едином стилистическом ключе. Начертания различаются насыщенностью штрихов (толщиной линий знака) — от сверхтонкого до сверхжирного — и наклоном знаков, формирующим прямое, курсивное (с изменённым рисунком букв) или наклонное (линейно наклонённое прямое) написание. Одна гарнитура может насчитывать от единственного начертания до нескольких десятков вариантов, что позволяет

выделять структурные элементы документа — заголовки, подзаголовки, основной текст.

Шрифты классифицируются по конструктивным особенностям строения знаков. Антиквенные шрифты, также называемые серифными (*от serif*), имеют характерные засечки на концах штрихов. Гротески, или рубленые шрифты (*sans serif fonts*), лишены засечек и отличаются геометрической простотой форм. Моноширинные шрифты (*monospace fonts*) характеризуются одинаковой шириной всех знаков, что делает их незаменимыми в программировании и табличной вёрстке. Акцидентные шрифты предназначены для заголовков, логотипов и декоративных целей, тогда как рукописные имитируют различные виды ручного письма. Шрифты также характеризуются стилистическими особенностями: геометрические построены на основе простых форм, гуманистические отличаются плавными формами, имитирующими ручное письмо пером, брусковые отличаются прямоугольными засечками и одинаковыми толщинами со штрихами.

Важной характеристикой компьютерного шрифта является поддержка языков и письменностей. Глифовый (*от glyph, графическое представление символа*) состав определяет, какие наборы символов способен отображать шрифт. Базовая латиница охватывает западноевропейские языки, расширенная латиница добавляет знаки для центральноевропейских и балтийских языков, кириллица обеспечивает поддержку славянских языков. Системы письма различаются также направлением чтения: латиница и кириллица читаются слева направо, арабское и еврейское письмо — справа налево, традиционные восточноазиатские тексты могут располагаться вертикально сверху вниз. Направление письма влияет на технические требования к отображению шрифта и должно учитываться при организации шрифтовой коллекции.

Компьютерные шрифты существуют в различных технических форматах. Формат TrueType, разработанный компанией Apple в конце восьмидесятых годов, получил широкое распространение благодаря поддержке со стороны операционной системы Windows. Формат OpenType, созданный совместными усилиями Microsoft и Adobe, расширил возможности шрифтов за счёт поддержки дополнительных типографических функций и большего количества глифов. Веб-шрифты в форматах WOFF и WOFF2 оптимизированы для передачи по сети и используются на веб-страницах.

Отображение шрифта в цифровой среде требует постоянного пересчёта. При изменении размера кегля, ширины текстового блока или масштаба страницы программа заново вычисляет положение каждого символа, переносы слов, межстрочные и межбуквенные интервалы. Этот процесс происходит в текстовых редакторах, издательских системах и веб-браузерах. Чтобы сделать пересёт гибче, существуют вариативные

шрифты (*variable fonts*). Они расширяют возможности динамической адаптации, позволяя плавно изменять насыщенность, ширину и другие параметры начертания без загрузки отдельных файлов.

Электронная типография предполагает наличие парка рабочих станций, на которых выполняются задачи вёрстки, дизайна и допечатной подготовки. Рабочая станция представляет собой персональный компьютер с определёнными аппаратными характеристиками: процессором, объёмом оперативной памяти, типом и ёмкостью накопителя, параметрами графического адаптера. На каждой станции установлена операционная система — Windows, macOS или Linux — определённой версии. Сочетание аппаратных характеристик и операционной системы определяет, какие форматы шрифтов поддерживаются и насколько корректно они отображаются.

Шрифты в электронной типографии — лицензируемые продукты. Лицензия определяет допустимые способы использования: некоторые шрифты распространяются бесплатно для любых целей, другие требуют приобретения коммерческой лицензии, третьи доступны по подписке. Лицензионные условия могут ограничивать количество рабочих станций, на которых разрешена установка шрифта, тираж печатной продукции или число просмотров веб-страниц. Без учёта лицензий типография рискует нарушить авторские права.

В электронной типографии работают несколько категорий специалистов. Системный администратор отвечает за техническое состояние парка рабочих станций, установку операционных систем, настройку программного обеспечения и распределение шрифтовых ресурсов между станциями. Дизайнер осуществляет подбор шрифтов для проектов, формирует запросы на приобретение новых гарнитур и использует установленные шрифты в своей работе. Менеджер по закупкам ведёт учёт лицензий, оформляет приобретение новых шрифтов, контролирует сроки действия подписок и соответствие количества установок лицензионным ограничениям.

Процесс пополнения шрифтовой коллекции начинается с выявления потребности. Дизайнер, работая над проектом, может обнаружить необходимость в гарнитуре, отсутствующей в текущей коллекции типографии. Он формирует запрос, указывая название шрифта, требуемые начертания и обоснование выбора. Менеджер по закупкам проверяет наличие шрифта у издательств, сравнивает условия лицензирования и стоимость. После согласования бюджета оформляется приобретение лицензии. Системный администратор получает файлы шрифта и выполняет установку на рабочие станции, для которых предназначена лицензия.

Допечатная подготовка и печать требуют строгого соответствия шрифтов на всех этапах производства. Верстальщик создаёт макет на своей рабочей станции, используя определённые гарнитуры и начертания. За-

тем файл передаётся на допечатную подготовку, затем на печать. На каждом этапе программа обращается к шрифтам, установленным локально. Если на одной из станций нужный шрифт отсутствует, система подставит замену — похожий шрифт с другими метриками. Это приведёт к смещению строк, изменению интервалов, нарушению переносов и выравнивания. Ошибка может остаться незамеченной до печати тиража, а исправление потребует перевёрстки и дополнительных затрат. Централизованный учёт шрифтов позволяет контролировать идентичность коллекций на всех станциях и предотвращать подобные ситуации ещё на этапе передачи макета.

Процесс инвентаризации шрифтовых ресурсов направлен на поддержание актуальности сведений о коллекции. Системный администратор периодически проводит сверку фактически установленных шрифтов на рабочих станциях с данными учётной системы. Выявляются расхождения: шрифты, установленные без учёта, удалённые шрифты, изменения в конфигурации станций. По результатам инвентаризации учётные данные корректируются. Менеджер по закупкам проверяет соответствие количества установок лицензионным ограничениям и при необходимости приобретает дополнительные лицензии или деактивирует избыточные установки.

Таким образом, учёт компьютерных шрифтов в электронной типографии представляет собой комплексную задачу, охватывающую систематизацию сведений о шрифтовых гарнитурах и их характеристиках, регистрацию парка рабочих станций с их аппаратно-программными конфигурациями, отслеживание лицензионных условий и сроков их действия, а также фиксацию связей между конкретными шрифтами и станциями, на которых они установлены. Организация такого учёта обеспечивает соблюдение лицензионных требований, оперативный поиск необходимых шрифтов и рациональное использование типографических ресурсов.

1.2 Цели функционирования базы данных

1. Хранение сведений о шрифтовых гарнитурах, их начертаниях и характеристиках.
2. Хранение информации о рабочих станциях и их аппаратно-программных конфигурациях.
3. Хранение информации о лицензиях на шрифты, условиях использования и сроках действия.
4. Хранение информации об установленных шрифтах на каждой рабочей станции.

1.3 Польза от внедрения базы данных

Польза от внедрения базы данных напрямую следует из поставленных целей. Единый каталог гарнитур позволяет быстро находить нужный шрифт и проверять его характеристики. Реестр рабочих станций с информацией об установленных шрифтах даёт возможность убедиться, что на всех станциях установлены одни и те же шрифты. Учёт лицензий предотвращает нарушение авторских прав и помогает планировать бюджет на продление подписок. Фиксация связей между шрифтами и станциями упрощает выявление расхождений. В результате типография получает инструмент, обеспечивающий согласованность шрифтовых коллекций, соблюдение лицензионных требований и прозрачность использования типографических ресурсов.

1.4 Описание сущностей

1. Гарнитура — это семейство начертаний шрифта, объединённых общим стилем и предназначенных для совместного использования в типографических работах.

Атрибуты: название, год создания, описание, число начертаний.

2. Начертание — это конкретный вариант гарнитуры, определяемый насыщенностью штрихов и наклоном знаков, хранящийся в отдельном файле шрифта.

Атрибуты: название, насыщенность, наклон, имя файла, размер файла.

3. Категория — это классификационная группа, объединяющая шрифты по конструктивным особенностям строения знаков (например, по наличию засечек или ширине символов).

Атрибуты: название, описание, иконка, число гарнитур.

4. Формат — это технический стандарт хранения шрифтовых данных, определяющий структуру файла, способ описания знаков и совместимость с платформами.

Атрибуты: название, расширение, год появления, описание, поддержка веба.

5. Язык — это система письма с определённым набором символов и направлением чтения, поддержка которой обеспечивается наличием соответствующих глифов в начертании шрифта.

Атрибуты: название, код ISO, направление письма, тип письменности.

6. Символ — это отдельный знак, поддерживаемый конкретным начертанием шрифта для определённого языка. Такая сущность позволяет проверять наличие конкретного символа и подсчитывать число символов по начертаниям, форматам, категориям и языкам.

Атрибуты: значение символа, код Unicode, число начертаний, число языков. Число начертаний и число языков являются вычисляемыми атрибутами.

7. Лицензия — это документ, устанавливающий права и ограничения на использование шрифта, включая количество рабочих станций и срок действия; на каждую гарнитуру распространяются отдельные лицензии.

Атрибуты: тип, срок действия, максимальное число станций, стоимость, дата приобретения.

8. Издательство — это организация, занимающаяся разработкой, производством и распространением шрифтов, а также предоставлением лицензий на их использование.

Атрибуты: название, страна, сайт, год основания, контактный email.

9. Рабочая станция — это персональный компьютер в составе парка типографии, используемый для выполнения задач вёрстки, дизайна и допечатной подготовки.

Атрибуты: инвентарный номер, процессор, объём памяти, тип накопителя, операционная система.

10. Операционная система — это комплекс программ, управляющий аппаратными ресурсами рабочей станции и обеспечивающий среду для работы прикладного программного обеспечения.

Атрибуты: название, версия, разрядность, год выпуска.

11. Использование — это факт размещения конкретного начертания шрифта на рабочей станции, фиксирующий связь между шрифтовым ресурсом и оборудованием.

Атрибуты: дата установки, дата удаления, путь к файлу, статус, дата последнего использования.

1.5 Описание ролей и субъектов

Системный администратор обслуживает парк рабочих станций. Он устанавливает операционные системы, настраивает программное обеспечение и распределяет шрифты между станциями.

Дизайнер использует шрифты в своей работе. Он подбирает гарнитуры для проектов и формирует запросы на приобретение новых шрифтов.

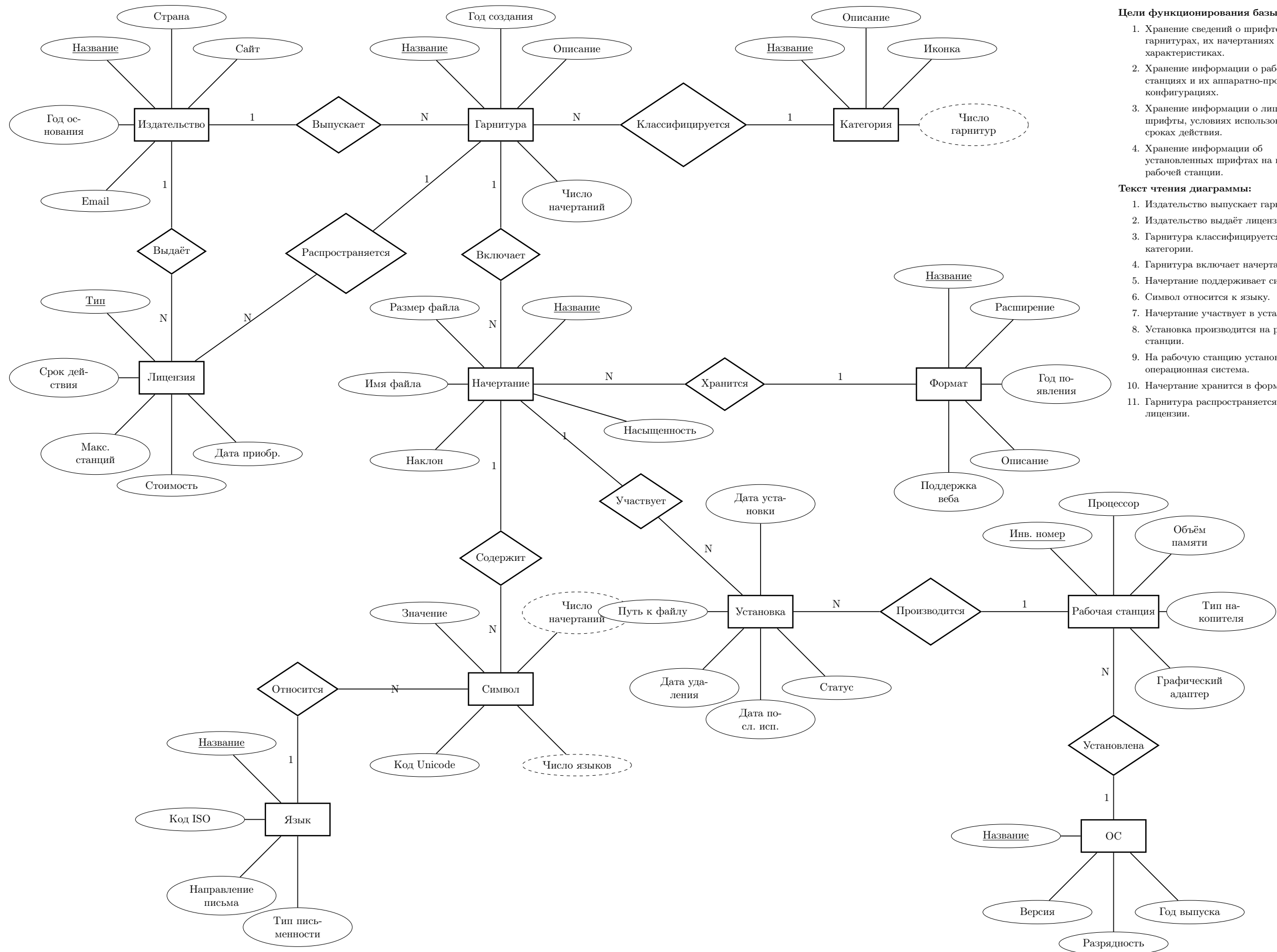
Менеджер по закупкам ведёт учёт лицензий. Он оформляет приобретение шрифтов, контролирует сроки действия подписок и соответствие количества установок лицензионным ограничениям.

Издательство выпускает шрифты и предоставляет лицензии на их использование. Типография приобретает лицензии у издательств и устанавливает шрифты на рабочие станции.

1.6 ER-диаграмма

На рисунке 1 (стр. 10) представлена ER-диаграмма предметной области. Перечень связей:

1. Издательство выпускает гарнитуру.
2. Издательство выдаёт лицензию.
3. Гарнитура классифицируется по категории.
4. Гарнитура включает начертание.
5. Начертание поддерживает символ.
6. Символ относится к языку.
7. Начертание участвует в установке.
8. Установка производится на рабочей станции.
9. На рабочую станцию установлена операционная система.
10. Начертание хранится в формате.
11. Гарнитура распространяется по лицензии.



Цели функционирования базы данных:

1. Хранение сведений о шрифтовых гарнитурах, их начертаниях и характеристиках.
2. Хранение информации о рабочих станциях и их аппаратно-программных конфигурациях.
3. Хранение информации о лицензиях на шрифты, условиях использования и сроках действия.
4. Хранение информации об установленных шрифтах на каждой рабочей станции.

Текст чтения диаграммы:

1. Издательство выпускает гарнитуру.
2. Издательство выдаёт лицензию.
3. Гарнитура классифицируется по категории.
4. Гарнитура включает начертание.
5. Начертание поддерживает символ.
6. Символ относится к языку.
7. Начертание участвует в установке.
8. Установка производится на рабочей станции.
9. На рабочую станцию установлена операционная система.
10. Начертание хранится в формате.
11. Гарнитура распространяется по лицензии.

Рис. 1: ER-диаграмма предметной области «Учёт компьютерных шрифтов в электронной типографии»

1.7 Схема объектов

Схема объектов представлена на рисунке 2.

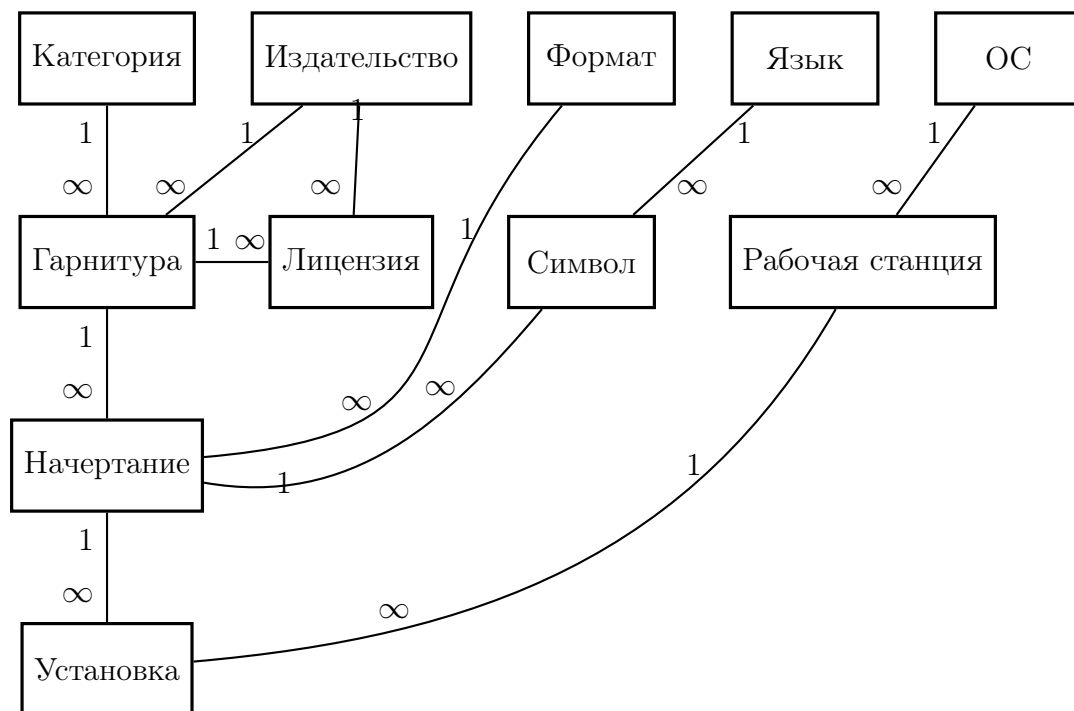


Рис. 2: Схема объектов предметной области «Учёт компьютерных шрифтов в электронной типографии»

2 Проектирование

Схема базы данных приведена на рисунках 3 и 4 на русском и английском языках соответственно.

2.1 Лингвистическое описание схемы базы данных

1. *Издательство* (*id_Издательство* **INT**, *Название* **VARCHAR(100)**, *Страна* **VARCHAR(60)**, *Сайт* **TEXT**, *Год_основания* **SMALLINT**, *Email* **VARCHAR(254)**).
2. *Гарнитура* (*id_Гарнитура* **INT**, *Название* **VARCHAR(100)**, *Год_создания* **SMALLINT**, *Описание* **TEXT**, *Число_начертаний* **SMALLINT**, *id_Издательство* **INT**, *id_Категория* **INT**).
3. *Категория* (*id_Категория* **INT**, *Название* **VARCHAR(100)**, *Описание* **TEXT**, *Иконка* **TEXT**, *Число_гарнитур* **INT**).
4. *Лицензия* (*id_Лицензия* **INT**, *Тип* **VARCHAR(30)**, *Срок_действия* **DATE**, *Максимальное_число_станций* **SMALLINT**, *Стоимость* **DECIMAL(10,2)**, *Дата_приобретения* **DATE**, *id_Издательство* **INT**, *id_Гарнитура* **INT**).
5. *Формат* (*id_Формат* **INT**, *Название* **VARCHAR(100)**, *Расширение* **VARCHAR(10)**, *Год_появления* **SMALLINT**, *Описание* **TEXT**, *Поддержка_веба* **BOOLEAN**).
6. *Начертание* (*id_Начертание* **INT**, *Название* **VARCHAR(100)**, *Насыщенность* **VARCHAR(30)**, *Наклон* **VARCHAR(30)**, *Размер_файла* **INT**, *id_Гарнитура* **INT**, *id_Формат* **INT**).
7. *Язык* (*id_Язык* **INT**, *Название* **VARCHAR(60)**, *Код_ISO* **CHAR(3)**, *Направление_письма* **VARCHAR(30)**, *Тип_письменности* **VARCHAR(30)**).
8. *Символ* (*id_Символ* **INT**, *Значение* **VARCHAR(8)**, *Код_Unicode* **VARCHAR(10)**, *id_Начертание* **INT**, *id_Язык* **INT**).
9. *Рабочая_станция* (*id_Рабочая_станция* **INT**, *Инвентарный_номер* **VARCHAR(20)**, *Процессор* **VARCHAR(80)**, *Объём_памяти* **INT**, *Тип_накопителя* **VARCHAR(30)**, *id_ОС* **INT**).
10. *ОС* (*id_ОС* **INT**, *Название* **VARCHAR(50)**, *Версия* **VARCHAR(30)**, *Разрядность* **SMALLINT**, *Год_выпуска* **SMALLINT**).
11. *Установка* (*id_Установка* **INT**, *Дата_установки* **DATE**, *Дата_удаления* **DATE**, *Путь_к_файлу* **TEXT**, *Статус* **VARCHAR(30)**, *Дата_последнего_использования* **DATE**, *id_Начертание* **INT**, *id_Рабочая_станция* **INT**).

Для запросов, связанных с наличием конкретных символов и подсчётом числа символов, сущность *Символ* хранит отдельные знаки, поддерживаемые начертанием для языка. Такая структура позволяет искать начертания по конкретному символу и строить агрегаты по числу символов.

2.2 Описание таблиц базы данных

В таблицах 1–11 представлены атрибуты проектируемой базы данных. В полях таблиц перечислены: ключ (*тип ключа «PK» — Primary Key, «FK» — Foreign Key*), названия атрибутов на русском и на английском языках, тип атрибута и ссылка на другую таблицу, если таковая имеется.

Таблица 1: Издательство — Publisher

Издательство — Publisher						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Publisher	id_Издательство	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Country	Страна	VARCHAR(60)	NOT NULL	—
4	—	Website	Сайт	TEXT	NULL	—
5	—	Foundation_year	Год_основания	SMALLINT	NOT NULL	—
6	—	Email	Email	VARCHAR(254)	NULL	—

В таблице 1 для URL-адресов выбран тип **TEXT**, так как считаем, что в окне браузера можно ввести URL-адрес любой длины. Тип **VARCHAR(254)** выбран для email-адресов. Их максимальная длина составляет 254 символа согласно RFC 5321¹. **VARCHAR(100)** — для имён гарнитур и начертаний² (аналогично в таблицах 2, 3, 5 и 6) тогда как **VARCHAR(60)** ограничивает названия стран в соответствии с максимальной длиной официальных наименований. Для стоимости лицензии применяется **DECIMAL(10,2)**, обеспечивающий точные денежные вычисления без ошибок округления. Код языка хранится в **CHAR(3)**, так как коды ISO 639³ имеют фиксированную длину в два или три символа.

¹RFC 5321, Section 4.5.3.1 — <https://datatracker.ietf.org/doc/html/rfc5321#section-4.5.3.1>

²Например, самое длинное название начертания, которое удалось найти «Noto Sans Devanagari SemiCondensed ExtraLight Italic», — 52 символа.

³<https://www.iso.org/iso-639-language-code>

Таблица 2: Гарнитура — Font_family

<i>Гарнитура — Font_family</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Font_family	id_Гарнитура	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Year_created	Год_создания	SMALLINT	NULL	—
4	—	Description	Описание	TEXT	NULL	—
5	—	Typeface_count	Число_начертаний	SMALLINT	NOT NULL	—
6	FK	id_Publisher	id_Издательство	INT	NOT NULL	Publisher.id_Publisher
7	FK	id_Category	id_Категория	INT	NOT NULL	Category.id_Category

Таблица 3: Категория — Category

<i>Категория — Category</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Category	id_Категория	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Description	Описание	TEXT	NULL	—
4	—	Icon	Иконка	TEXT	NULL	—
5	—	Font_family_count	Число_гарнитур	INT	NOT NULL	—

В таблицах 3 и 11 имя директории имеет тип TEXT, т.к. может быть любой длины в рамках операционной системы.

Таблица 4: Лицензия — License

<i>Лицензия — License</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_License	id_Лицензия	INT	NOT NULL	—
2	—	Type	Тип	VARCHAR(30)	NOT NULL	—
3	—	Expiration_date	Срок_действия	DATE	NULL	—
4	—	Max_stations	Макс_число_станций	SMALLINT	NULL	—
5	—	Cost	Стоимость	DECIMAL(10,2)	NOT NULL	—
6	—	Purchase_date	Дата_приобретения	DATE	NOT NULL	—
7	FK	id_Publisher	id_Издательство	INT	NOT NULL	Publisher.id_Publisher
8	FK	id_Font_family	id_Гарнитура	INT	NOT NULL	Font_family.id_Font_family

В таблице 4 тип лицензии хранится в `VARCHAR(30)`, что достаточно для значений вроде «OFL», «Commercial», «Subscription» или «Trial». Поле `Срок_действия` допускает `NULL`, поскольку бессрочные лицензии не имеют даты окончания. Аналогично `Макс_число_станций` допускает `NULL` для лицензий без ограничения на количество установок.

Таблица 5: Формат — Format

<i>Формат — Format</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Format	id_Формат	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Extension	Расширение	VARCHAR(10)	NOT NULL	—
4	—	Year_released	Год_появления	SMALLINT	NULL	—
5	—	Description	Описание	TEXT	NULL	—
6	—	Web_support	Поддержка_веба	BOOLEAN	NOT NULL	—

В таблице 5 расширение файла хранится в `VARCHAR(10)`, что покрывает существующие форматы шрифтов: «.ttf», «.otf», «.woff2».

Таблица 6: Начертание — Typeface

<i>Начертание — Typeface</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Typeface	id_Начертание	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Weight	Насыщенность	VARCHAR(30)	NOT NULL	—
4	—	Slope	Наклон	VARCHAR(30)	NOT NULL	—
5	—	File_size	Размер_файла	INT	NOT NULL	—
6	FK	id_Font_family	id_Гарнитура	INT	NOT NULL	Font_family.id_Font_family
7	FK	id_Format	id_Формат	INT	NOT NULL	Format.id_Format

В таблице 6 насыщенность и наклон хранятся в `VARCHAR(30)`, что достаточно для стандартных значений: «Thin», «ExtraLight», «Regular», «Bold», «ExtraBold» для насыщенности и «Upright», «Italic», «Oblique» для наклона. Размер файла хранится в `INT` и измеряется в байтах — максимальное значение около 2 ГБ, что с запасом покрывает любые файлы шрифтов.

Таблица 7: Язык — Language

<i>Язык — Language</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Language	id_Язык	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(60)	NOT NULL	—
3	—	ISO_code	Код_ISO	CHAR(3)	NOT NULL	—
4	—	Writing_direction	Направление_письма	VARCHAR(30)	NOT NULL	—
5	—	Script_type	Тип_письменности	VARCHAR(30)	NOT NULL	—

В таблице 7 название языка хранится в `VARCHAR(60)`, что покрывает самые длинные наименования в реестре ISO 639-3⁴, например «Min Nan Chinese» или «Ancient Greek». Направление письма хранится в `VARCHAR(30)` для значений «left-to-right», «right-to-left» или «top-to-bottom». Тип письменности — «alphabetic», «syllabic», «logographic» — также укладывается в `VARCHAR(30)`. В таблице 8 значение символа хранится в `VARCHAR(8)`, чтобы учитывать символы, представленные несколькими байтами или

⁴ISO 639-3 Code Tables — https://iso639-3.sil.org/code_tables/639/data

составными последовательностями Unicode. Поле Код_Unicode хранится в `VARCHAR(10)` для обозначений вида «U+0041». В таблице 11 поле Статус принимает значения «active», «disabled» или «removed» — тоже используется тип `VARCHAR(30)`.

Таблица 8: Символ — Symbol

<i>Символ — Symbol</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Symbol	id_Символ	INT	NOT NULL	—
2	—	Value	Значение	VARCHAR(8)	NOT NULL	—
3	—	Unicode_code	Код_Unicode	VARCHAR(10)	NOT NULL	—
4	FK	id_Typeface	id_Начертание	INT	NOT NULL	Typeface.id_Typeface
5	FK	id_Language	id_Язык	INT	NOT NULL	Language.id_Language

Таблица 9: Рабочая станция — Workstation

<i>Рабочая станция — Workstation</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Workstation	id_Рабочая_станция	INT	NOT NULL	—
2	—	Inventory_number	Инвентарный_номер	VARCHAR(20)	NOT NULL	—
3	—	Processor	Процессор	VARCHAR(80)	NOT NULL	—
4	—	Memory_size	Объём_памяти	INT	NOT NULL	—
5	—	Storage_type	Тип_накопителя	VARCHAR(30)	NOT NULL	—
6	FK	id_OS	id_ОС	INT	NOT NULL	OS.id_OS

В таблице 9 название процессора хранится в `VARCHAR(80)`: согласно спецификации Intel⁵, название процессора занимает до 48 символов, а запас до 80 учитывает возможные дополнения при ручном вводе. Объём памяти хранится в `INT` и измеряется в мегабайтах. Тип накопителя — `VARCHAR(30)` для значений «SSD», «HDD» или «NVMe SSD». Инвентарный номер — `VARCHAR(20)`, поскольку инвентарные номера обычно содержат буквенно-цифровые коды фиксированной длины, считаем, что в нашей типографии максимальная длина кода — 20 символов.

⁵Intel® Processor Identification and the CPUID Instruction, Application Note 485, Section 3.2.3 — <https://datasheets.chipdb.org/Intel/x86/CPUID/24161831.pdf>

Таблица 10: Операционная система — OS

<i>Операционная система — OS</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_OS	id_ОС	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(50)	NOT NULL	—
3	—	Version	Версия	VARCHAR(30)	NOT NULL	—
4	—	Bitness	Разрядность	SMALLINT	NOT NULL	—
5	—	Release_year	Год_выпуска	SMALLINT	NOT NULL	—

В таблице 10 название операционной системы хранится в `VARCHAR(50)`: согласно реестру IANA⁶, официальное имя ОС может содержать до 40 символов, а запас до 50 учитывает неофициальные наименования. Версия — в `VARCHAR(30)`, поскольку версии могут содержать буквенно-цифровые обозначения, например «24H2», «15.4.1» или «24.04 LTS». Разрядность хранится в `SMALLINT` и принимает значения 32 или 64.

Таблица 11: Установка — Installation

<i>Установка — Installation</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Installation	id_Установка	INT	NOT NULL	—
2	—	Installation_date	Дата_установки	DATE	NOT NULL	—
3	—	Removal_date	Дата_удаления	DATE	NULL	—
4	—	File_path	Путь_к_файлу	TEXT	NOT NULL	—
5	—	Status	Статус	VARCHAR(30)	NOT NULL	—
6	—	Last_used_date	Дата_последнего_использования	DATE	NULL	—
7	FK	id_Typeface	id_Начертание	INT	NOT NULL	Typeface.id_Typeface
8	FK	id_Workstation	id_Рабочая_станция	INT	NOT NULL	Workstation.id_Workstation

⁶IANA Operating System Names — <https://www.iana.org/assignments/operating-system-names>

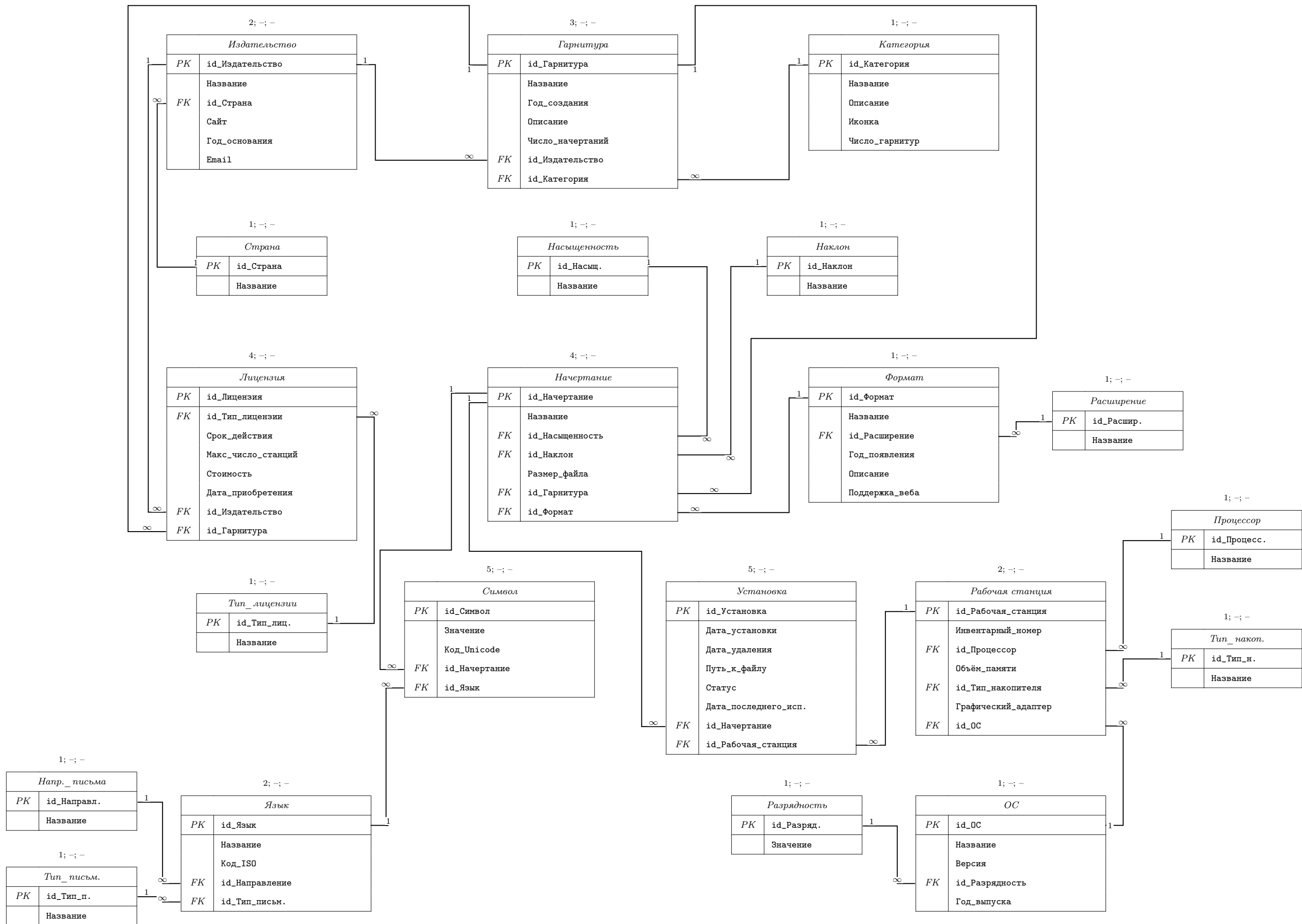


Рис. 3: Схема базы данных на русском языке

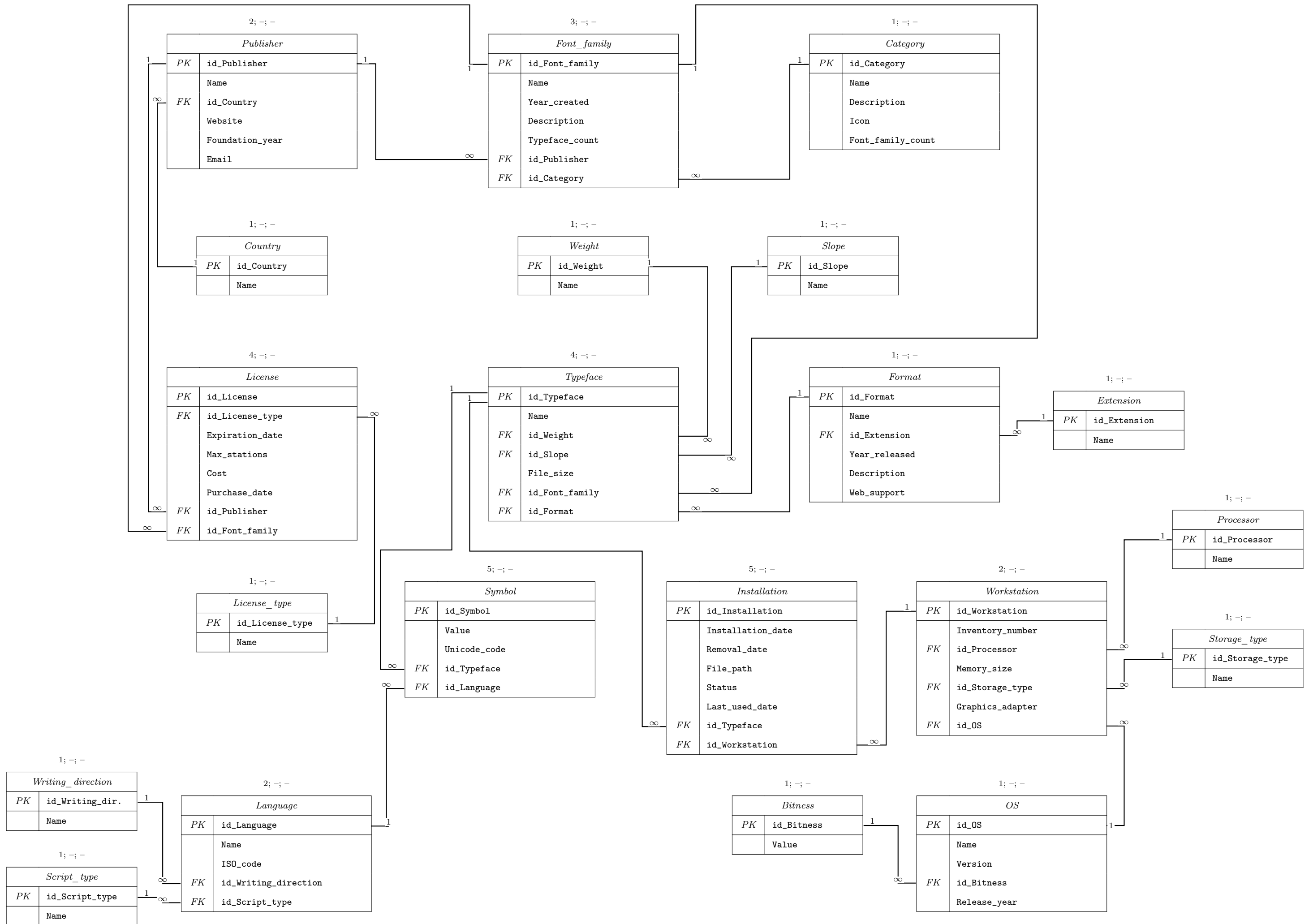


Рис. 4: Схема базы данных на английском языке

2.3 Создание таблиц базы данных и генерация записей

Для создания таблиц базы данных был написан SQL-скрипт для СУБД PostgreSQL ([приложение А](#)). В проекте используется база данных `typography`, запущенная в контейнере Docker. Контейнер базы данных создаётся на основе образа PostgreSQL и доступен на локальном порту 5432.

В указанном выше SQL-скрипте последовательно создаются таблицы предметной области: `Category`, `Font_family`, `Format`, `Typeface`, `Language` и `Symbol`. Порядок создания таблиц выбран с учётом внешних ключей: сначала создаются справочные и независимые таблицы, затем таблицы, которые на них ссылаются.

Структура связей между таблицами, используемыми при программной реализации базы данных, представлена на уменьшенных схемах базы данных на рисунках [5](#) и [6](#).

Для заполнения базы данных тестовыми данными используется программа на TypeScript ([приложение Б](#)). Подключение к базе данных выполняется при помощи библиотеки `pg`. Для генерации правдоподобных значений используется библиотека `@faker-js/faker`. Запуск скриптов выполняется в среде Node.js с использованием `tsx`.

Генерация записей должна выполняться в том же порядке, что и создание зависимостей между таблицами. Сначала заполняются таблицы `Category`, `Format` и `Language`, так как они не зависят от других таблиц. Затем создаются записи в таблице `Font_family`, для каждой из которых выбирается существующая категория. После этого создаются записи в таблице `Typeface`, где для каждого начертания выбираются существующие гарнитура и формат. Последней заполняется таблица `Symbol`, связывающая существующие языки, начертания и поддерживаемые символы.

Количество записей в каждой таблице приведено на схемах базы данных (рисунки [5](#) и [6](#)).

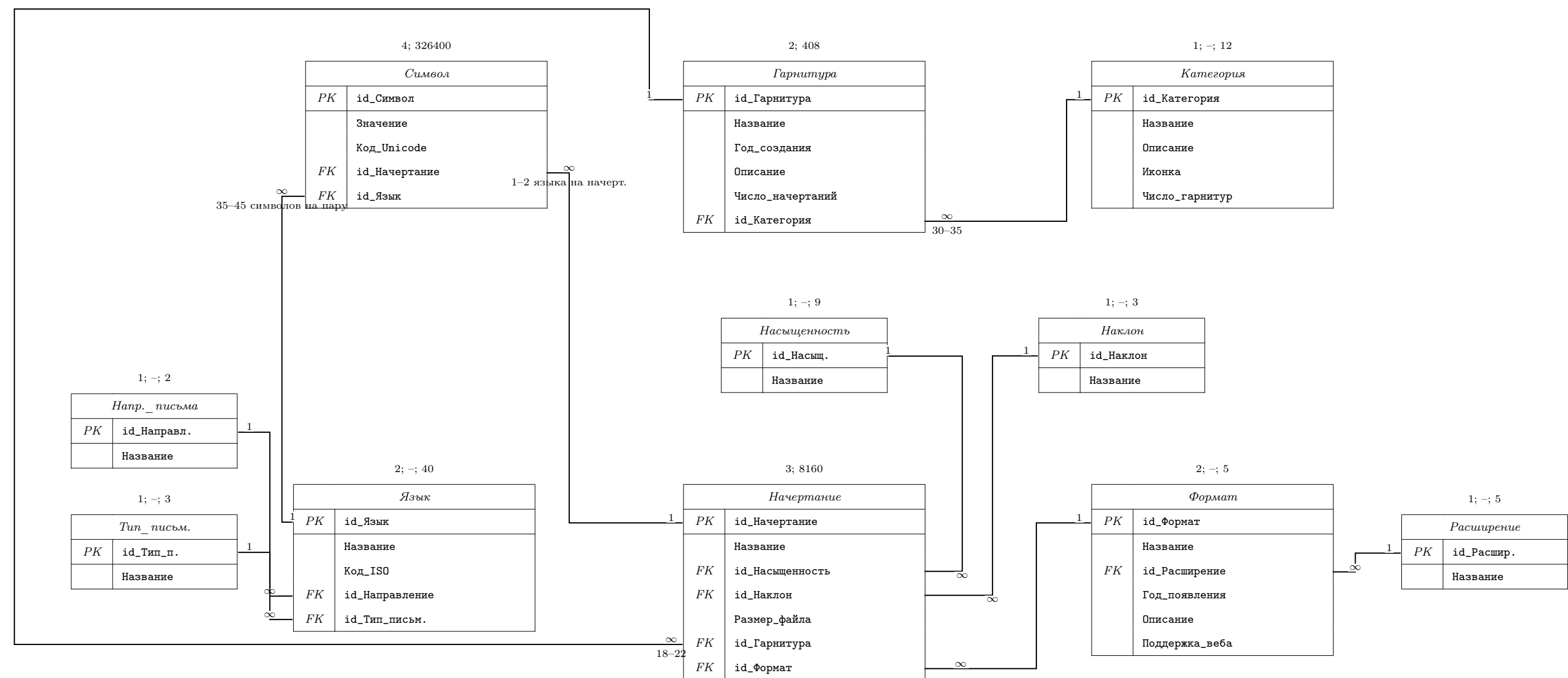


Рис. 5: Программируемая схема базы данных

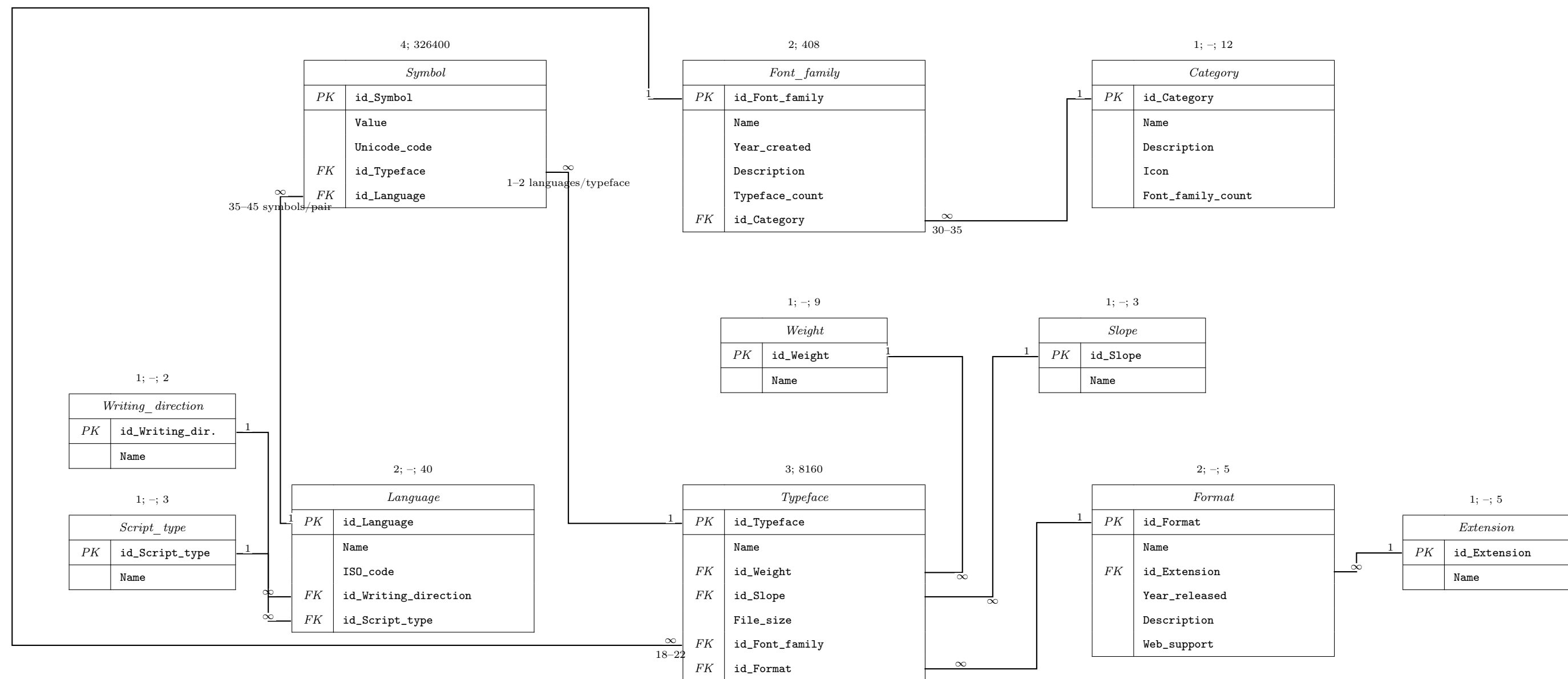


Рис. 6: Программируемая схема базы данных на английском языке

3 Программирование

Приложение А. SQL-скрипт создания таблиц базы данных

```
1 DROP TABLE IF EXISTS Symbol CASCADE;
2
3 DROP TABLE IF EXISTS Typeface CASCADE;
4
5 DROP TABLE IF EXISTS Font_family CASCADE;
6
7 DROP TABLE IF EXISTS Language CASCADE;
8
9 DROP TABLE IF EXISTS Format CASCADE;
10
11 DROP TABLE IF EXISTS Category CASCADE;
12
13 DROP TABLE IF EXISTS Weight CASCADE;
14
15 DROP TABLE IF EXISTS Slope CASCADE;
16
17 DROP TABLE IF EXISTS Extension CASCADE;
18
19 DROP TABLE IF EXISTS ISO_code CASCADE;
20
21 DROP TABLE IF EXISTS Writing_direction CASCADE;
22
23 DROP TABLE IF EXISTS Script_type CASCADE;
24
25 CREATE TABLE Category (
26     id_Category SERIAL PRIMARY KEY,
27     Name VARCHAR(100) NOT NULL,
28     Description TEXT,
29     Icon TEXT,
30     Font_family_count INT NOT NULL DEFAULT 0
31 );
32
33 CREATE TABLE Weight (
34     id_Weight SERIAL PRIMARY KEY,
35     Name VARCHAR(30) NOT NULL UNIQUE
36 );
37
38 CREATE TABLE Slope (
39     id_Slope SERIAL PRIMARY KEY,
40     Name VARCHAR(30) NOT NULL UNIQUE
41 );
42
43 CREATE TABLE Extension (
44     id_Extension SERIAL PRIMARY KEY,
45     Name VARCHAR(10) NOT NULL UNIQUE
46 );
47
48 CREATE TABLE Writing_direction (
49     id_Writing_direction SERIAL PRIMARY KEY,
50     Name VARCHAR(30) NOT NULL UNIQUE
51 );
52
53 CREATE TABLE Script_type (
54     id_Script_type SERIAL PRIMARY KEY,
55     Name VARCHAR(30) NOT NULL UNIQUE
56 );
57
58 CREATE TABLE Format (
59     id_Format SERIAL PRIMARY KEY,
60     Name VARCHAR(100) NOT NULL,
61     id_Extension INT NOT NULL REFERENCES Extension(id_Extension),
62     Year_released SMALLINT,
63     Description TEXT,
64     Web_support BOOLEAN NOT NULL
65 );
66
67 CREATE TABLE Language (
68     id_Language SERIAL PRIMARY KEY,
69     Name VARCHAR(60) NOT NULL,
70     ISO_code CHAR(3) NOT NULL UNIQUE,
71     id_Writing_direction INT NOT NULL REFERENCES Writing_direction(id_Writing_direction),
72     id_Script_type INT NOT NULL REFERENCES Script_type(id_Script_type)
```

```

73 );
74
75 CREATE TABLE Font_family (
76     id_Font_family SERIAL PRIMARY KEY,
77     Name VARCHAR(100) NOT NULL,
78     Year_created SMALLINT,
79     Description TEXT,
80     Typeface_count SMALLINT NOT NULL DEFAULT 0,
81     id_Category INT NOT NULL REFERENCES Category(id_Category)
82 );
83
84 CREATE TABLE Typeface (
85     id_Typeface SERIAL PRIMARY KEY,
86     Name VARCHAR(100) NOT NULL,
87     id_Weight INT NOT NULL REFERENCES Weight(id_Weight),
88     id_Slope INT NOT NULL REFERENCES Slope(id_Slope),
89     File_size INT NOT NULL,
90     id_Font_family INT NOT NULL REFERENCES Font_family(id_Font_family),
91     id_Format INT NOT NULL REFERENCES Format(id_Format)
92 );
93
94 CREATE TABLE Symbol (
95     id_Symbol SERIAL PRIMARY KEY,
96     Value VARCHAR(8) NOT NULL,
97     Unicode_code VARCHAR(10) NOT NULL,
98     id_Typeface INT NOT NULL REFERENCES Typeface(id_Typeface),
99     id_Language INT NOT NULL REFERENCES Language(id_Language),
100
101     CONSTRAINT uq_symbol_value_typeface_language
102         UNIQUE (Value, id_Typeface, id_Language)
103 );

```

Приложение Б. Программа на TypeScript для заполнения базы данных тестовыми данными

```
1 // db.ts
2 import pg from "pg";
3
4 export const pool = new pg.Pool({
5   host: "localhost",
6   port: 5432,
7   database: "typography",
8   user: "ilyasemenov",
9   password: "type",
10 });
11
12
13 // init.ts
14 import { readFileSync } from "fs";
15 import { pool } from "./db.js";
16
17 async function init() {
18   const sql = readFileSync("init.sql", "utf-8");
19   await pool.query(sql);
20   console.log("Tables created successfully");
21   await pool.end();
22 }
23
24 init().catch(console.error);
25
26
27 // constants.ts
28 import { faker } from "@faker-js/faker";
29
30 export const BASE_COUNTS = {
31   categories: 12,
32   formats: 5,
33   languages: 40,
34 } as const;
35
36 export const COEFFICIENT_LIMITS = {
37   fontFamiliesPerCategory: { min: 30, max: 35 },
38   typefacesPerFamily: { min: 18, max: 22 },
39   languagesPerTypeface: { min: 1, max: 2 },
40   symbolsPerTypefaceLanguage: { min: 35, max: 45 },
41 } as const;
42
43 export const COEFFICIENTS = {
44   fontFamiliesPerCategory: faker.number.int(
45     COEFFICIENT_LIMITS.fontFamiliesPerCategory,
46   ),
47
48   typefacesPerFamily: faker.number.int(COEFFICIENT_LIMITS.typefacesPerFamily),
49
50   languagesPerTypeface: faker.number.int(
51     COEFFICIENT_LIMITS.languagesPerTypeface,
52   ),
53
54   symbolsPerTypefaceLanguage: faker.number.int(
55     COEFFICIENT_LIMITS.symbolsPerTypefaceLanguage,
56   ),
57 } as const;
58
59 export const COUNTS = {
60   categories: BASE_COUNTS.categories,
61   formats: BASE_COUNTS.formats,
62   languages: BASE_COUNTS.languages,
63
64   fontFamilies: BASE_COUNTS.categories * COEFFICIENTS.fontFamiliesPerCategory,
65
66   typefaces:
67     BASE_COUNTS.categories *
68     COEFFICIENTS.fontFamiliesPerCategory *
69     COEFFICIENTS.typefacesPerFamily,
70
71   symbols:
72     BASE_COUNTS.categories *
```

```

73     COEFFICIENTS.fontFamiliesPerCategory *
74     COEFFICIENTS.typefacesPerFamily *
75     COEFFICIENTS.languagesPerTypeface *
76     COEFFICIENTS.symbolsPerTypefaceLanguage,
77 } as const;
78 export const categoryNames = [
79     "Serif",
80     "Sans-serif",
81     "Monospace",
82     "Display",
83     "Handwriting",
84     "Slab-serif",
85     "Geometric",
86     "Humanist",
87     "Grotesque",
88     "Neo-grotesque",
89     "Transitional",
90     "Didone",
91 ];
92
93 export const categoryDescs: Record<string, string> = {
94     Serif: "Typefaces with decorative strokes at the ends of letterforms",
95     "Sans-serif": "Typefaces without decorative strokes",
96     Monospace: "Typefaces where each character occupies the same width",
97     Display: "Typefaces designed for headlines and large sizes",
98     Handwriting: "Typefaces imitating handwritten text",
99     "Slab-serif": "Typefaces with thick rectangular serifs",
100    Geometric: "Sans-serif typefaces based on geometric shapes",
101    Humanist: "Typefaces with organic strokes inspired by calligraphy",
102    Grotesque: "Early sans-serif typefaces with irregular proportions",
103    "Neo-grotesque": "Refined sans-serif typefaces with uniform strokes",
104    Transitional: "Serif typefaces bridging old-style and modern designs",
105    Didone: "High-contrast serif typefaces with thin hairlines",
106 };
107
108 export const formats = [
109     {
110         name: "TrueType",
111         ext: ".ttf",
112         year: 1991,
113         desc: "Scalable font format developed by Apple",
114         web: false,
115     },
116     {
117         name: "OpenType",
118         ext: ".otf",
119         year: 1996,
120         desc: "Scalable font format by Microsoft and Adobe",
121         web: false,
122     },
123     {
124         name: "WOFF",
125         ext: ".woff",
126         year: 2010,
127         desc: "Web Open Font Format",
128         web: true,
129     },
130     {
131         name: "WOFF2",
132         ext: ".woff2",
133         year: 2014,
134         desc: "Compressed Web Open Font Format",
135         web: true,
136     },
137     {
138         name: "TrueType Collection",
139         ext: ".ttc",
140         year: 1994,
141         desc: "Multiple TrueType fonts in a single file",
142         web: false,
143     },
144 ];
145
146 export const languages = [
147     {
148         name: "English",
149         iso: "eng",
150         writingDirection: "left-to-right",

```

```

151     scriptType: "alphabetic",
152 },
153 {
154     name: "Russian",
155     iso: "rus",
156     writingDirection: "left-to-right",
157     scriptType: "alphabetic",
158 },
159 {
160     name: "German",
161     iso: "deu",
162     writingDirection: "left-to-right",
163     scriptType: "alphabetic",
164 },
165 {
166     name: "French",
167     iso: "fra",
168     writingDirection: "left-to-right",
169     scriptType: "alphabetic",
170 },
171 {
172     name: "Spanish",
173     iso: "spa",
174     writingDirection: "left-to-right",
175     scriptType: "alphabetic",
176 },
177 {
178     name: "Italian",
179     iso: "ita",
180     writingDirection: "left-to-right",
181     scriptType: "alphabetic",
182 },
183 {
184     name: "Portuguese",
185     iso: "por",
186     writingDirection: "left-to-right",
187     scriptType: "alphabetic",
188 },
189 {
190     name: "Dutch",
191     iso: "nld",
192     writingDirection: "left-to-right",
193     scriptType: "alphabetic",
194 },
195 {
196     name: "Polish",
197     iso: "pol",
198     writingDirection: "left-to-right",
199     scriptType: "alphabetic",
200 },
201 {
202     name: "Czech",
203     iso: "ces",
204     writingDirection: "left-to-right",
205     scriptType: "alphabetic",
206 },
207 {
208     name: "Slovak",
209     iso: "slk",
210     writingDirection: "left-to-right",
211     scriptType: "alphabetic",
212 },
213 {
214     name: "Ukrainian",
215     iso: "ukr",
216     writingDirection: "left-to-right",
217     scriptType: "alphabetic",
218 },
219 {
220     name: "Belarusian",
221     iso: "bel",
222     writingDirection: "left-to-right",
223     scriptType: "alphabetic",
224 },
225 {
226     name: "Serbian",
227     iso: "srp",
228     writingDirection: "left-to-right",

```

```

229     scriptType: "alphabetic",
230 },
231 {
232     name: "Croatian",
233     iso: "hrv",
234     writingDirection: "left-to-right",
235     scriptType: "alphabetic",
236 },
237 {
238     name: "Bulgarian",
239     iso: "bul",
240     writingDirection: "left-to-right",
241     scriptType: "alphabetic",
242 },
243 {
244     name: "Greek",
245     iso: "ell",
246     writingDirection: "left-to-right",
247     scriptType: "alphabetic",
248 },
249 {
250     name: "Turkish",
251     iso: "tur",
252     writingDirection: "left-to-right",
253     scriptType: "alphabetic",
254 },
255 {
256     name: "Finnish",
257     iso: "fin",
258     writingDirection: "left-to-right",
259     scriptType: "alphabetic",
260 },
261 {
262     name: "Swedish",
263     iso: "swe",
264     writingDirection: "left-to-right",
265     scriptType: "alphabetic",
266 },
267 {
268     name: "Norwegian",
269     iso: "nor",
270     writingDirection: "left-to-right",
271     scriptType: "alphabetic",
272 },
273 {
274     name: "Danish",
275     iso: "dan",
276     writingDirection: "left-to-right",
277     scriptType: "alphabetic",
278 },
279 {
280     name: "Icelandic",
281     iso: "isl",
282     writingDirection: "left-to-right",
283     scriptType: "alphabetic",
284 },
285 {
286     name: "Irish",
287     iso: "gle",
288     writingDirection: "left-to-right",
289     scriptType: "alphabetic",
290 },
291 {
292     name: "Welsh",
293     iso: "cym",
294     writingDirection: "left-to-right",
295     scriptType: "alphabetic",
296 },
297 {
298     name: "Arabic",
299     iso: "ara",
300     writingDirection: "right-to-left",
301     scriptType: "alphabetic",
302 },
303 {
304     name: "Hebrew",
305     iso: "heb",
306     writingDirection: "right-to-left",

```

```

307     scriptType: "alphabetic",
308 },
309 {
310     name: "Persian",
311     iso: "fas",
312     writingDirection: "right-to-left",
313     scriptType: "alphabetic",
314 },
315 {
316     name: "Urdu",
317     iso: "urd",
318     writingDirection: "right-to-left",
319     scriptType: "alphabetic",
320 },
321 {
322     name: "Hindi",
323     iso: "hin",
324     writingDirection: "left-to-right",
325     scriptType: "syllabic",
326 },
327 {
328     name: "Bengali",
329     iso: "ben",
330     writingDirection: "left-to-right",
331     scriptType: "syllabic",
332 },
333 {
334     name: "Tamil",
335     iso: "tam",
336     writingDirection: "left-to-right",
337     scriptType: "syllabic",
338 },
339 {
340     name: "Telugu",
341     iso: "tel",
342     writingDirection: "left-to-right",
343     scriptType: "syllabic",
344 },
345 {
346     name: "Thai",
347     iso: "tha",
348     writingDirection: "left-to-right",
349     scriptType: "syllabic",
350 },
351 {
352     name: "Lao",
353     iso: "lao",
354     writingDirection: "left-to-right",
355     scriptType: "syllabic",
356 },
357 {
358     name: "Chinese",
359     iso: "zho",
360     writingDirection: "left-to-right",
361     scriptType: "logographic",
362 },
363 {
364     name: "Japanese",
365     iso: "jpn",
366     writingDirection: "left-to-right",
367     scriptType: "logographic",
368 },
369 {
370     name: "Korean",
371     iso: "kor",
372     writingDirection: "left-to-right",
373     scriptType: "syllabic",
374 },
375 {
376     name: "Vietnamese",
377     iso: "vie",
378     writingDirection: "left-to-right",
379     scriptType: "alphabetic",
380 },
381 {
382     name: "Indonesian",
383     iso: "ind",
384     writingDirection: "left-to-right",

```

```

385     scriptType: "alphabetic",
386   },
387 ];
388
389 export const weights = [
390   "Thin",
391   "ExtraLight",
392   "Light",
393   "Regular",
394   "Medium",
395   "SemiBold",
396   "Bold",
397   "ExtraBold",
398   "Black",
399 ];
400
401 export const slopes = ["Upright", "Italic", "Oblique"];
402
403 export const tables = [
404   "Category",
405   "Format",
406   "Language",
407   "Font_family",
408   "Typeface",
409   "Symbol",
410 ];
411
412
413 // seed.ts
414 import { faker } from "@faker-js/faker";
415 import { pool } from "./db.js";
416 import {
417   BASE_COUNTS,
418   COEFFICIENTS,
419   COUNTS,
420   categoryNames,
421   categoryDescs,
422   formats,
423   languages,
424   weights,
425   slopes,
426   tables,
427 } from "./constants.js";
428 import type { PoolClient } from "pg";
429
430 type IdMap = Map<string, number>;
431
432 type TypefaceRow = {
433   id: number;
434   fontFamilyId: number;
435 };
436
437 class Seeder {
438   private client!: PoolClient;
439
440   private categoryIds: number[] = [];
441   private formatIds: number[] = [];
442   private languageIds: number[] = [];
443   private fontFamilyIds: number[] = [];
444   private typefaces: TypefaceRow[] = [];
445
446   private weightIds: IdMap = new Map();
447   private slopeIds: IdMap = new Map();
448   private extensionIds: IdMap = new Map();
449   private writingDirectionIds: IdMap = new Map();
450   private scriptTypeIds: IdMap = new Map();
451
452   async run() {
453     this.client = await pool.connect();
454
455     try {
456       await this.client.query("BEGIN");
457
458       await this.clearTables();
459
460       await this.seedDictionaries();
461
462       await this.seedCategories();

```



```

463     await this.seedFormats();
464     await this.seedLanguages();
465
466     await this.seedFontFamilies();
467     await this.seedTypefaces();
468     await this.seedSymbols();
469
470     await this.updateCategoryCounts();
471
472     await this.client.query("COMMIT");
473
474     console.log("\nSeeding complete!");
475     await this.printSummary();
476   } catch (error) {
477     await this.client.query("ROLLBACK");
478     throw error;
479   } finally {
480     this.client.release();
481     await pool.end();
482   }
483 }
484
485 private async clearTables() {
486   await this.client.query(`
487     TRUNCATE TABLE
488       Symbol,
489       Typeface,
490       Font_family,
491       Language,
492       Format,
493       Category,
494       Weight,
495       Slope,
496       Extension,
497       Writing_direction,
498       Script_type
499     RESTART IDENTITY CASCADE
500   `);
501
502   console.log("Tables cleared");
503 }
504
505 private async seedDictionaries() {
506   await this.seedWeightDictionary();
507   await this.seedSlopeDictionary();
508   await this.seedExtensionDictionary();
509   await this.seedWritingDirectionDictionary();
510   await this.seedScriptTypeDictionary();
511 }
512
513 private async seedWeightDictionary() {
514   for (const name of weights) {
515     const result = await this.client.query(
516       `
517         INSERT INTO Weight (Name)
518         VALUES ($1)
519         RETURNING id_Weight
520       `,
521       [name],
522     );
523
524     this.weightIds.set(name, result.rows[0].id_weight);
525   }
526
527   console.log(`Weight: ${this.weightIds.size}`);
528 }
529
530 private async seedSlopeDictionary() {
531   for (const name of slopes) {
532     const result = await this.client.query(
533       `
534         INSERT INTO Slope (Name)
535         VALUES ($1)
536         RETURNING id_Slope
537       `,
538       [name],
539     );
540

```

```

541     this.slopeIds.set(name, result.rows[0].id_slope);
542 }
543
544 console.log(`Slope: ${this.slopeIds.size}`);
545 }
546
547 private async seedExtensionDictionary() {
548     const uniqueExtensions = [...new Set(formats.map((format) => format.ext))];
549
550     for (const extension of uniqueExtensions) {
551         const result = await this.client.query(
552             `
553             INSERT INTO Extension (Name)
554             VALUES ($1)
555             RETURNING id_Extension
556             `,
557             [extension],
558         );
559
560         this.extensionIds.set(extension, result.rows[0].id_extension);
561     }
562
563     console.log(`Extension: ${this.extensionIds.size}`);
564 }
565
566 private async seedWritingDirectionDictionary() {
567     const selectedLanguages = languages.slice(0, BASE_COUNTS.languages);
568     const uniqueWritingDirections = [
569         ...new Set(
570             selectedLanguages.map((language) => language.writingDirection),
571         ),
572     ];
573
574     for (const name of uniqueWritingDirections) {
575         const result = await this.client.query(
576             `
577             INSERT INTO Writing_direction (Name)
578             VALUES ($1)
579             RETURNING id_Writing_direction
580             `,
581             [name],
582         );
583
584         this.writingDirectionIds.set(name, result.rows[0].id_writing_direction);
585     }
586
587     console.log(`Writing_direction: ${this.writingDirectionIds.size}`);
588 }
589
590 private async seedScriptTypeDictionary() {
591     const selectedLanguages = languages.slice(0, BASE_COUNTS.languages);
592     const uniqueScriptTypes = [
593         ...new Set(selectedLanguages.map((language) => language.scriptType)),
594     ];
595
596     for (const name of uniqueScriptTypes) {
597         const result = await this.client.query(
598             `
599             INSERT INTO Script_type (Name)
600             VALUES ($1)
601             RETURNING id_Script_type
602             `,
603             [name],
604         );
605
606         this.scriptTypeIds.set(name, result.rows[0].id_script_type);
607     }
608
609     console.log(`Script_type: ${this.scriptTypeIds.size}`);
610 }
611
612 private async seedCategories() {
613     const selectedCategories = categoryNames.slice(0, BASE_COUNTS.categories);
614
615     for (const name of selectedCategories) {
616         const result = await this.client.query(
617             `
618             INSERT INTO Category

```

```

619         (Name, Description, Icon, Font_family_count)
620     VALUES
621         ($1, $2, $3, 0)
622     RETURNING id_Category
623     `,
624     [name, categoryDescs[name], `/icons/${this.slugify(name)}.svg`],
625 );
626
627     this.categoryIds.push(result.rows[0].id_category);
628 }
629
630 console.log(`Category: ${this.categoryIds.length}`);
631 }
632
633 private async seedFormats() {
634     const selectedFormats = formats.slice(0, BASE_COUNTS.formats);
635
636     for (const format of selectedFormats) {
637         const extensionId = this.getRequiredId(
638             this.extensionIds,
639             format.ext,
640             "Extension",
641         );
642
643         const result = await this.client.query(
644             `
645             INSERT INTO Format
646             (Name, id_Extension, Year_released, Description, Web_support)
647             VALUES
648             ($1, $2, $3, $4, $5)
649             RETURNING id_Format
650             `,
651             [format.name, extensionId, format.year, format.desc, format.web],
652         );
653
654         this.formatIds.push(result.rows[0].id_format);
655     }
656
657     console.log(`Format: ${this.formatIds.length}`);
658 }
659
660 private async seedLanguages() {
661     const selectedLanguages = languages.slice(0, BASE_COUNTS.languages);
662
663     for (const language of selectedLanguages) {
664         const writingDirectionId = this.getRequiredId(
665             this.writingDirectionIds,
666             language.writingDirection,
667             "Writing_direction",
668         );
669
670         const scriptTypeId = this.getRequiredId(
671             this.scriptTypeIds,
672             language.scriptType,
673             "Script_type",
674         );
675
676         const result = await this.client.query(
677             `
678             INSERT INTO Language
679             (
680                 Name,
681                 ISO_code,
682                 id_Writing_direction,
683                 id_Script_type
684             )
685             VALUES
686             ($1, $2, $3, $4)
687             RETURNING id_Language
688             `,
689             [language.name, language.iso, writingDirectionId, scriptTypeId],
690         );
691
692         this.languageIds.push(result.rows[0].id_language);
693     }
694
695     console.log(`Language: ${this.languageIds.length}`);
696 }

```

```

697
698 private async seedFontFamilies() {
699     let created = 0;
700
701     const batchSize = 1000;
702     let values: unknown[] = [];
703     let placeholders: string[] = [];
704
705     for (const categoryId of this.categoryIds) {
706         for (let i = 0; i < COEFFICIENTS.fontFamiliesPerCategory; i++) {
707             const index = values.length;
708
709             placeholders.push(
710                 `(${index + 1}, ${index + 2}, ${index + 3}, ${index + 4}, ${index + 5})`,
711             );
712
713             values.push(
714                 this.makeFontFamilyName(created),
715                 faker.number.int({ min: 1950, max: 2024 }),
716                 faker.lorem.sentence(),
717                 COEFFICIENTS.typefacesPerFamily,
718                 categoryId,
719             );
720
721             created++;
722
723             if (placeholders.length >= batchSize) {
724                 await this.insertFontFamilyBatch(placeholders, values);
725                 values = [];
726                 placeholders = [];
727             }
728         }
729
730         console.log(
731             `Font_family by category: ${this.categoryIds.indexOf(categoryId) +
732             ↵ 1}/${this.categoryIds.length}`,
733         );
734     }
735
736     if (placeholders.length > 0) {
737         await this.insertFontFamilyBatch(placeholders, values);
738     }
739
740     console.log(`Font_family: ${this.fontFamilyIds.length}`);
741 }
742
743 private async insertFontFamilyBatch(
744     placeholders: string[],
745     values: unknown[],
746 ) {
747     const result = await this.client.query(
748         `
749         INSERT INTO Font_family
750         (
751             Name,
752             Year_created,
753             Description,
754             Typeface_count,
755             id_Category
756         )
757         VALUES
758             ${placeholders.join(", ")}
759         RETURNING id_Font_family
760         `,
761         values,
762     );
763
764     for (const row of result.rows) {
765         this.fontFamilyIds.push(row.id_font_family);
766     }
767 }
768
769 private async seedTypefaces() {
770     let created = 0;
771
772     const batchSize = 1000;
773     let values: unknown[] = [];

```

```

773 let placeholders: string[] = [];
774
775 for (const fontFamilyId of this.fontFamilyIds) {
776   const familyName = this.makeTypefaceFamilyPrefix(fontFamilyId);
777
778   for (let i = 0; i < COEFFICIENTS.typefacesPerFamily; i++) {
779     const weight = weights[i % weights.length];
780     const slope = slopes[i % slopes.length];
781
782     const weightId = this.getRequiredId(this.weightIds, weight, "Weight");
783     const slopeId = this.getRequiredId(this.slopeIds, slope, "Slope");
784
785     const formatId = this.formatIds[i % this.formatIds.length];
786
787     const typefaceName =
788       slope === "Upright"
789         ? `${familyName} ${weight}`
790         : `${familyName} ${weight} ${slope}`;
791
792     const index = values.length;
793
794     placeholders.push(
795       `($${index + 1}, $${index + 2}, $${index + 3}, $${index + 4}, $${index + 5}, $${index + 6})`,
796     );
797
798     values.push(
799       typefaceName.slice(0, 100),
800       weightId,
801       slopeId,
802       faker.number.int({ min: 30_000, max: 400_000 }),
803       fontFamilyId,
804       formatId,
805     );
806
807     created++;
808
809     if (placeholders.length >= batchSize) {
810       await this.insertTypefaceBatch(placeholders, values);
811       values = [];
812       placeholders = [];
813
814       if (created % 5000 === 0) {
815         console.log(`Typeface: $${created}/$${COUNTS.typefaces}`);
816       }
817     }
818   }
819 }
820
821 if (placeholders.length > 0) {
822   await this.insertTypefaceBatch(placeholders, values);
823 }
824
825 console.log(`Typeface: $${this.typefaces.length}`);
826 }
827
828 private async insertTypefaceBatch(placeholders: string[], values: unknown[]) {
829   const result = await this.client.query(
830     `
831     INSERT INTO Typeface
832     (
833       Name,
834       id_Weight,
835       id_Slope,
836       File_size,
837       id_Font_family,
838       id_Format
839     )
840     VALUES
841       $${placeholders.join(", ")}
842     RETURNING id_Typeface, id_Font_family
843     `,
844     values,
845   );
846
847   for (const row of result.rows) {
848     this.typefaces.push({
849       id: row.id_typeface,

```

```

850         fontFamilyId: row.id_font_family,
851     });
852 }
853 }
854
855 private async seedSymbols() {
856     let created = 0;
857
858     const symbolRows = this.buildSymbolRows();
859
860     const batchSize = 5000;
861     let values: unknown[] = [];
862     let placeholders: string[] = [];
863
864     for (const row of symbolRows) {
865         const index = values.length;
866         const symbol = this.makeSymbol(row.languageIndex, row.symbolOffset);
867
868         placeholders.push(
869             `(${index + 1}, ${index + 2}, ${index + 3}, ${index + 4})`,
870         );
871
872         values.push(
873             symbol.value,
874             symbol.unicodeCode,
875             row.typefaceId,
876             row.languageId,
877         );
878
879         created++;
880
881         if (placeholders.length >= batchSize) {
882             await this.insertSymbolBatch(placeholders, values);
883             values = [];
884             placeholders = [];
885
886             if (created % 50_000 === 0) {
887                 console.log(`Symbol: ${created}/${symbolRows.length}`);
888             }
889         }
890     }
891
892     if (placeholders.length > 0) {
893         await this.insertSymbolBatch(placeholders, values);
894     }
895
896     console.log(`Symbol: ${created}`);
897 }
898
899 private buildSymbolRows(): Array<{
900     typefaceId: number;
901     languageId: number;
902     languageIndex: number;
903     symbolOffset: number;
904 }> {
905     const languagesPerTypeface = COEFFICIENTS.languagesPerTypeface;
906     const symbolsPerTypefaceLanguage = COEFFICIENTS.symbolsPerTypefaceLanguage;
907
908     if (this.typefaces.length === 0) {
909         throw new Error("Cannot seed Symbol: no typefaces");
910     }
911
912     if (this.languageIds.length === 0) {
913         throw new Error("Cannot seed Symbol: no languages");
914     }
915
916     if (languagesPerTypeface > this.languageIds.length) {
917         throw new Error(
918             `Cannot seed Symbol: languagesPerTypeface=${languagesPerTypeface}, but
919             ↪ languages=${this.languageIds.length}`,
920         );
921     }
922
923     const totalSymbolRows =
924         this.typefaces.length *
925         languagesPerTypeface *
926         symbolsPerTypefaceLanguage;

```

```

926
927 if (totalSymbolRows !== COUNTS.symbols) {
928   console.warn(
929     `Symbol target mismatch: generated ${totalSymbolRows}, constants contain ${COUNTS.symbols}`,
930   );
931 }
932
933 const rows: Array<{
934   typefaceId: number;
935   languageId: number;
936   languageIndex: number;
937   symbolOffset: number;
938 }> = [];
939
940 for (
941   let typefaceIndex = 0;
942   typefaceIndex < this.typefaces.length;
943   typefaceIndex++
944 ) {
945   const typeface = this.typefaces[typefaceIndex];
946
947   for (let offset = 0; offset < languagesPerTypeface; offset++) {
948     const languageIndex =
949       (typefaceIndex + offset) % this.languageIds.length;
950
951     const languageId = this.languageIds[languageIndex];
952
953     for (
954       let symbolOffset = 0;
955       symbolOffset < symbolsPerTypefaceLanguage;
956       symbolOffset++
957     ) {
958       rows.push({
959         typefaceId: typeface.id,
960         languageId,
961         languageIndex,
962         symbolOffset,
963       });
964     }
965   }
966 }
967
968 if (rows.length !== totalSymbolRows) {
969   throw new Error(
970     `Cannot seed Symbol: created ${rows.length}/${totalSymbolRows} rows`,
971   );
972 }
973
974 return rows;
975 }
976
977 private async insertSymbolBatch(placeholders: string[], values: unknown[]) {
978   await this.client.query(
979     `
980     INSERT INTO Symbol
981     (
982       Value,
983       Unicode_code,
984       id_Typeface,
985       id_Language
986     )
987     VALUES
988     ${placeholders.join(", ")}
989     `,
990     values,
991   );
992 }
993
994 private async updateCategoryCounts() {
995   await this.client.query(
996     `
997     UPDATE Category
998     SET Font_family_count = (
999       SELECT COUNT(*)
1000       FROM Font_family
1001       WHERE Font_family.id_Category = Category.id_Category
1002     );
1003 `

```

```

1004     console.log("Category counts updated");
1005 }
1006
1007 private makeFontFamilyName(index: number): string {
1008     const adjective = faker.word.adjective();
1009     const noun = faker.word.noun();
1010
1011     const generated = `${adjective} ${noun}`
1012         .split(" ")
1013         .map((part) => part.charAt(0).toUpperCase() + part.slice(1))
1014         .join(" ");
1015
1016     return `${generated} ${index + 1}`.slice(0, 100);
1017 }
1018
1019 private makeTypefaceFamilyPrefix(fontFamilyId: number): string {
1020     return `Family ${fontFamilyId}`;
1021 }
1022
1023 private makeSymbol(languageIndex: number, symbolOffset: number) {
1024     const codePoint = 0x0100 + languageIndex * 64 + symbolOffset;
1025     const value = String.fromCharCode(codePoint);
1026
1027     return {
1028         value,
1029         unicodeCode: `U+${codePoint.toString(16).toUpperCase().padStart(4, "0")}`,
1030     };
1031 }
1032
1033 private getRequiredId(map: IdMap, key: string, tableName: string): number {
1034     const id = map.get(key);
1035
1036     if (id === undefined) {
1037         throw new Error(`Missing dictionary value in ${tableName}: ${key}`);
1038     }
1039
1040     return id;
1041 }
1042
1043 private slugify(value: string): string {
1044     return value.toLowerCase().replaceAll(" ", "-").replaceAll("_", "-");
1045 }
1046
1047 private async printSummary() {
1048     let total = 0;
1049
1050     console.log("\nTable counts:");
1051
1052     for (const table of tables) {
1053         const result = await pool.query(`SELECT COUNT(*) FROM ${table}`);
1054         const count = Number(result.rows[0].count);
1055
1056         total += count;
1057         console.log(` ${table}: ${count}`);
1058     }
1059
1060     console.log(` TOTAL: ${total}`);
1061 }
1062 }
1063
1064 new Seeder().run().catch((error) => {
1065     console.error(error);
1066     process.exit(1);
1067 });

```